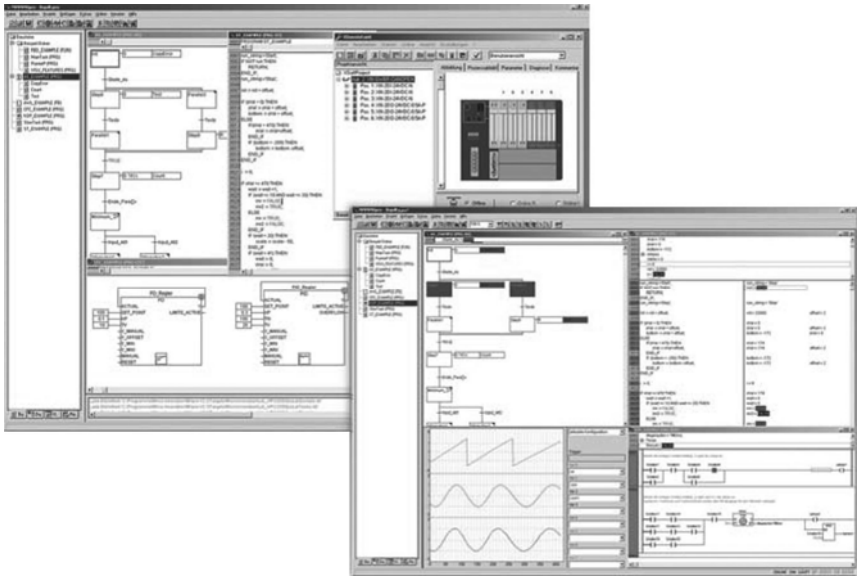
FtpClient Library



Powering Business Worldwide

Manufacturer

Eaton Automation AG
Spinnereistrasse 8-14
CH-9008 St. Gallen
Schweiz
www.eaton-automation.com
www.eaton.com

Support

Region North America
Eaton Corporation
Electrical Sector
1111 Superior Ave.
Cleveland, OH 44114
United States
877-ETN-CARE (877-386-2273)
www.eaton.com

Other regions
Please contact your local distributor or send an e-mail to: automation@eaton.com

Original instructions

German

Redaction

Daniel Lenherr

Brand and product names

All brand and product names are trademarks or registered trademarks of the owner concerned.

Copyright

© Eaton Automation AG, CH-9008 St. Gallen

All rights reserved, also for the translation.

None of this documents may be reproduced or processed, duplicated or distributed by electronic systems in any form (print, photocopy, microfilm or any other process) without the written permission of Eaton Automation AG, St. Gallen.

Subject to modifications.

Contents

1 General 4
1.1 Purpose of this document 4
1.2 Comments about this document 4
1.3 Additional documentation 4

2 FtpClient.lib 5
2.1 System requirements 5
2.1.1 PLC programming tool 5
2.1.2 PLC target system 5
2.2 Installation 5
2.2.1 PLC programming tool 5
2.2.2 PLC target system 5
2.3 Implementation and functionality 6
2.3.1 Behaviour on PLC stop 6
2.4 Overview 7
2.4.1 POUs 7
2.4.2 Data types 7
2.4.3 Global variables 7
2.5 POUs 8
2.5.1 FTP_OpenConnection (FB) 8
2.5.2 FTP_CloseConnection (FUN) 10
2.5.3 FTP_ReadDirectory (FB) 11
2.5.4 FTP_ChangeDirectory (FB) 13
2.5.5 FTP_GetCurrentDirectory (FB) 15
2.5.6 FTP_GetFile (FB) 17
2.5.7 FTP_PutFile (FB) 19
2.5.8 FTP_DeleteFile (FB) 21
2.5.9 FTP_GetSupplierVersion (FUN) 23
2.5.10 FTP_GetLastError (FUN) 23
2.6 Data types 24
2.7 Global variables 24

3 Appendix 25

1 General

1.1 Purpose of this document

This document describes the use of PLC libraries in the PLC programming tool "XSoft-CoDeSys-2" for Eaton Automation AG devices.

Library	Type	Description
FtpClient.lib	User library	Library for the data handling between several controls over FTP.

System libraries: Functions of system libraries must not be used direct by the User-PLC-Program. Functions of system libraries are necessary for basically control functions and must only merged in the User-PLC-program.

User libraries: Functions of user libraries can be used by the User-PLC-Program.

1.2 Comments about this document

Please send any comments, recommendations or suggestions relating to this document to automation@eaton.com.

1.3 Additional documentation

The following documents may be helpful in the use of the device in addition to this document. These can be downloaded from our home page (www.eaton-automation.com):

[1] -
User manual PLC programming tool CoDeSys V2.3

2 FtpClient.lib

2.1 System requirements

2.1.1 PLC programming tool

PLC programming tool	Version
XSoft-CoDeSys-2 resp. MXpro	≥ 2.3.6

2.1.2 PLC target system

PLC target system	PLC runtime system	Operating system (OS)
-	≥ PLCWinCE V 2.4.1 (x)	Windows CE Image Release ≥ 2.14.1 (x)

2.2 Installation

2.2.1 PLC programming tool

To use the library functions in the project, the library FtpClient.lib must be available on the programming-PC. With the installation of the according PLC programming tool the library will be installed by default.

Subsequently, the library can be added on the project with the library manager.

2.2.2 PLC target system

To use the library functions on the target system, the library functionalities must be available on the target system. The file PLCWinCECst.dll with the library functionalities is installed by default on according target system.

2.3 Implementation and functionality

In order to make possible FTP operation between the control serving as FTP client and a FTP server, the following steps are necessary:

- Activate the FTP server on corresponding FTP server
- Open the FTP connection to the FTP server

→ The processing of the functions and the function blocks can take some time.
For using the functions and the function blocks a low priority task with a low cycle time monitoring or no cycle time monitoring is to be defined.

2.3.1 Behaviour on PLC stop

On PLC stop all opened FTP connections are closed.

2.4 Overview

2.4.1 POUs

POU	Type	Description
FTP_OpenConnection	FB	Open a FTP connection
FTP_CloseConnection	FUN	Close a FTP connection
FTP_ReadDirectory	FB	Read the listing and file entries (directory and file names) of the current working directory on the ftp server
FTP_ChangeDirectory	FB	Change to a different working directory on the FTP server
FTP_GetCurrentDirectory	FB	Read the name of the current working directory on the FTP server
FTP_GetFile	FB	Transfer a file from the FTP server to the FTP Client and store local
FTP_PutFile	FB	Transfer a local file from the FTP client to the ftp server and store on the FTP server.
FTP_DeleteFile	FB	Delete a file on the FTP server
FTP_GetSupplierVersion	FUN	Read the library version
FTP_GetLastError	FUN	Read the error of the last FTP operation

2.4.2 Data types

Data type	Description
-	-

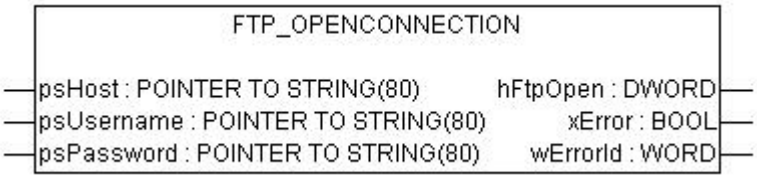
2.4.3 Global variables

Global variable	Data type	Description
FTP_gc_byVersion	BYTE	Constant 16#1 ... Specification version
FTP_gc_Version_1	BYTE	Constant with assigned variable name according to the constant value of the specification version

2.5 POUs

2.5.1 FTP_OpenConnection (FB)

This function block opens a FTP connection to the specified FTP server.



Input parameter	Data type	Description
psHost	POINTER TO STRING	Memory location (ADR(*xxx*)) of the string with the host name and/or IP address of the FTP server
psUsername	POINTER TO STRING	Memory location (ADR(*xxx*)) of the string with the user name or NULL
psPassword	POINTER TO STRING	Memory location (ADR(*xxx*)) of the string with the password or NULL

Output parameter	Data type	Description
hFtpOpen	DWORD	Handle of the FTP connection
xError	BOOL	FALSE … No error occured TRUE … Action abort by error
wErrorId	WORD	Additional error code Possible error codes are to be taken from the appendix (➔ Chap. **Fehler! Verweisquelle konnte nicht gefunden werden.**)

➔ With missing FTP server the process duration of the function block can take several seconds.

➔ This function block is called, processed and finished in a program cycle (similar to a function).

➔ This function block is not real time compatible. The processing of the function block can take some time.

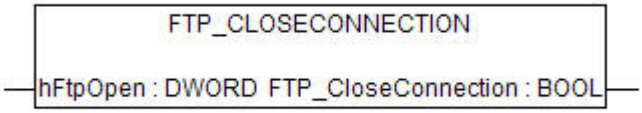
For using the function block a low priority task with a low cycle time monitoring or no cycle time monitoring is recommended.

Example:

```
VAR
    FtpOpenConnection           :       Ftp_OpenConnection;
    xFtpOpen                    :       BOOL;
    xFtpConnected               :       BOOL;
    sFtpHostname                :       STRING := '192.168.0.1';
    sFtpUsername                :       STRING := '';
    sFtpPassword                :       STRING := '';
    dwFtpHandle                 :       DWORD;
    dwLastError                 :       DWORD;
END_VAR

If xFtpOpen THEN
    xFtpOpen := FALSE;
    FtpOpenConnection    (
                            psHost          := ADR(sFtpHostname),
                            psUsername      := ADR(sFtpUsername),
                            psPassword      := ADR(sFtpPassword),
                            hFtpOpen        => dwFtpHandle,
                            xError          => ,
                            wErrorId        =>
                        );
    IF FtpOpenConnection.xError THEN
        (* FtpOpenConnection Error *)
        dwLastError := FTP_GetLastError( TRUE );
        xFtpConnected := FALSE;
    ELSE
        xFtpConnected := TRUE;
    END_IF;
END_IF;
```

2.5.2 FTP_CloseConnection (FUN)

This function closes a FTP connection to the specified FTP server.



Input parameter	Data type	Description
hFtpOpen	DWORD	Handle of the FTP connection (Output parameter *hFtpOpen* of FB *FTP_OpenConnection*))

Return value	Data type	Description
	BOOL	FALSE … Action successfully done TRUE … Action abort by error

→ This function is not real time compatible. The processing of the function can take some time. For using the function a low priority task with a low cycle time monitoring or no cycle time monitoring is recommended.

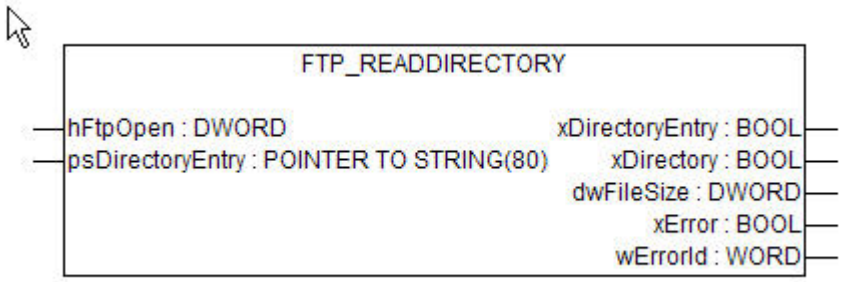
Example:

```
VAR
    xFtpClose          :    BOOL;
    xFtpConnected      :    BOOL;
    dwFtpHandle        :    DWORD;
    dwLastError        :    DWORD;
END_VAR

If xFtpClose THEN
    xFtpClose := FALSE;
    IF NOT FTP_CloseConnection( dwFtpHandle ) THEN
        dwLastError := FTP_GetLastError( TRUE );
    ELSE
        IF xFtpConnected THEN
            xFtpConnected := FALSE;
        END_IF;
    END_IF;
END_IF;
```

2.5.3 FTP_ReadDirectory (FB)

This function block reads the listing and file entries (directory and file names) of the current working directory on the ftp server.

In order to read all entries from the directory, the function block must be called so long, until xDirectoryEntry returns with FALSE!



Input parameter	Data type	Description
hFtpOpen	DWORD	Handle of the FTP connection (Output parameter *hFtpOpen* of FB *FTP_OpenConnection*)
psDirectoryEntry	POINTER TO STRING	Memory location (ADR(*xxx*)) of the string with the read directory and file names e.g. '/FtpServer'

Output parameter	Data type	Description
xDirectoryEntry	BOOL	FALSE ... no more entry available TRUE ... entry available
xDirectory	BOOL	FALSE ... read entry corresponds a file name TRUE ... read entry corresponds a directory name
dwFileSize	DWORD	With file name reads, it supplies the output of the file size in byte
xError	BOOL	FALSE ... No error occured TRUE ... Action abort by error
wErrorId	WORD	Additional error code Possible error codes are to be taken from the appendix (➔ Chap. **Fehler! Verweisquelle konnte nicht gefunden werden.**)

➔ This function block is called, processed and finished in a program cycle (similar to a function).

This function block is not real time compatible. The processing of the function block can take some time.

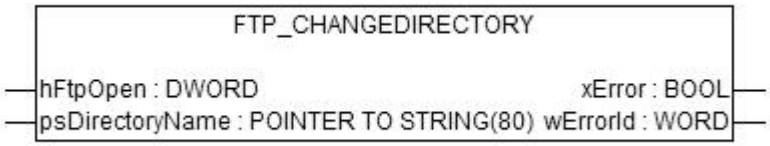
For using the function block a low priority task with a low cycle time monitoring or no cycle time monitoring is recommended.

Example:

```
VAR
    FtpReadDirectory            :   FTP_ReadDirectory;
    xFtpReadDirectory           :   BOOL;
    sReadDirectoryEntry         :   STRING;
    wDirectoryEntryCount        :   WORD;
    dwFtpHandle                 :   DWORD;
    dwLastError                 :   DWORD;
END_VAR

IF xFtpReadDirectory THEN
    xFtpReadDirectory := FALSE;
    wDirectoryEntryCount := 0;

    REPEAT
        FtpReadDirectory   (
                                hFtpOpen            :=      dwFtpHandle,
                                psDirectoryEntry    :=      ADR(sReadDirectoryEntry),
                                xError              => ,
                                wErrorId            =>
                            );
        IF FtpReadDirectory.xError THEN
            (* FtpReadDirectory Error *)
            dwLastError := FTP_GetLastError( TRUE );
            EXIT;
        ELSE
            IF FTPReadDirectory.xDirectoryEntry THEN
                wDirectoryEntryCount := wDirectoryEntryCount + 1;
            END_IF;
        END_IF;

    UNTIL FTPReadDirectory.xDirectoryEntry = FALSE
    END_REPEAT;

END_IF;
```

2.5.4 FTP_ChangeDirectory (FB)

This function block changes to a different working directory on the specified FTP server.



Input parameter	Data type	Description
hFtpOpen	DWORD	Handle of the FTP connection (Output parameter *hFtpOpen* of FB *FTP_OpenConnection*)
psDirectoryName	POINTER TO STRING	Memory location (ADR(*xxx*)) of the string with the directory name, into which should be changed e.g. '/FtpServer/'

Output parameter	Data type	Description
xError	BOOL	FALSE … No error occured TRUE … Action abort by error
wErrorId	WORD	Additional error code Possible error codes are to be taken from the appendix (➔ Chap. **Fehler! Verweisquelle konnte nicht gefunden werden.**)

➔ This function block is called, processed and finished in a program cycle (similar to a function).

➔ This function block is not real time compatible. The processing of the function block can take some time.

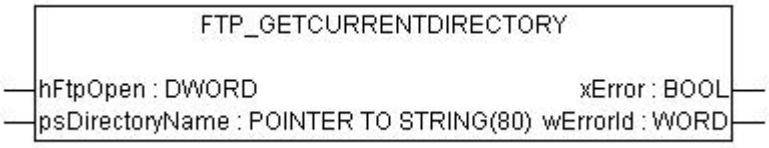
For using the function block a low priority task with a low cycle time monitoring or no cycle time monitoring is recommended.

Example:

```
VAR
    FtpChangeDirectory          :   FTP_ChangeDirectory;
    xFtpChangeDirectory         :   BOOL;
    sChangeDirectory            :   STRING := '/StorageCard';
    xFtpChangeDirectoryUp       :   BOOL;
    sChangeDirectoryUp          :   STRING := '..';
    dwFtpHandle                 :   DWORD;
    dwLastError                 :   DWORD;
END_VAR

IF xFtpChangeDirectory THEN
    xFtpChangeDirectory := FALSE;
    FtpChangeDirectory    (
                              hFtpOpen          := dwFtpHandle,
                              psDirectoryName   := ADR(sChangeDirectory),
                              xError            => ,
                              wErrorId          =>
                          );
    IF FtpChangeDirectory.xError THEN
        (* FtpChangeDirectory Error *)
        dwLastError := FTP_GetLastError( TRUE );
    END_IF;
END_IF;

IF xFtpChangeDirectoryUp THEN
    xFtpChangeDirectoryUp := FALSE;
    FtpChangeDirectory    (
                              hFtpOpen          := dwFtpHandle,
                              psDirectoryName   := ADR(sChangeDirectoryUp),
                              xError            => ,
                              wErrorId          =>
                              );
    IF FtpChangeDirectory.xError THEN
        (* FtpChangeDirectory Error *)
        dwLastError := FTP_GetLastError( TRUE );
    END_IF;
END_IF;
```

2.5.5 FTP_GetCurrentDirectory (FB)

This function block reads the name of the current working directory on the FTP server.



Input parameter	Data type	Description
hFtpOpen	DWORD	Handle of the FTP connection (Output parameter *hFtpOpen* of FB *FTP_OpenConnection*)
psDirectoryName	POINTER TO STRING	Memory location (ADR(*xxx*)) of the string with the directory name of the current working directory on the FTP server

Output parameter	Data type	Description
xError	BOOL	FALSE … No error occured TRUE … Action abort by error
wErrorId	WORD	Additional error code Possible error codes are to be taken from the appendix (➔ Chap. **Fehler! Verweisquelle konnte nicht gefunden werden.**)

➔ This function block is called, processed and finished in a program cycle (similar to a function).

➔ This function block is not real time compatible. The processing of the function block can take some time.

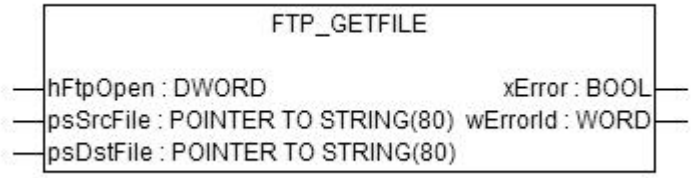
For using the function block a low priority task with a low cycle time monitoring or no cycle time monitoring is recommended.

Example:

```
VAR
    FtpGetCurrentDirectory          :       FTP_GetCurrentDirectory;
    xFtpGetCurrentDirectory         :       BOOL;
    sCurrentDirectory               :       STRING;
    dwFtpHandle                     :       DWORD;
    dwLastError                     :       DWORD;
END_VAR

IF xFtpGetCurrentDirectory THEN
    xFtpGetCurrentDirectory := FALSE;
    sCurrentDirectory := 'unknown';
    FtpGetCurrentDirectory      (
                                    hFtpOpen            := dwFtpHandle,
                                    psDirectoryName     := ADR(sCurrentDirectory),
                                    xError              => ,
                                    wErrorId            =>
                                );
    IF FtpGetCurrentDirectory.xError THEN
        (* FtpGetCurrentDirectory Error *)
        dwLastError := FTP_GetLastError( TRUE );
    END_IF;
END_IF;
```

2.5.6 FTP_GetFile (FB)

This function block transfers a file from the FTP server to the FTP client and stores it in a local file.



Input parameter	Data type	Description
hFtpOpen	DWORD	Handle of the FTP connection (Output parameter *hFtpOpen* of FB *FTP_OpenConnection*)
psSrcFile	POINTER TO STRING	Memory location (ADR(*xxx*)) of the string with the file name which is on the FTP server and which can be transferred. e.g. '/FtpServer/GetOut.txt'
psDstFile	POINTER TO STRING	Memory location (ADR(*xxx*)) of the string with the file name of the local and transferred file. e.g. '\Temp\GetIn.txt'

Output parameter	Data type	Description
xError	BOOL	FALSE … No error occured TRUE … Action abort by error
wErrorId	WORD	Additional error code Possible error codes are to be taken from the appendix (➔ Chap. **Fehler! Verweisquelle konnte nicht gefunden werden.**)

➔ This function block is called, processed and finished in a program cycle (similar to a function).

➔ This function block is not real time compatible. The processing of the function block can take some time.

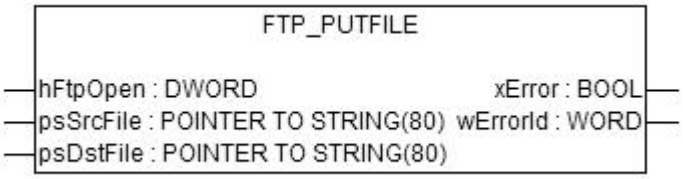
For using the function block a low priority task with a low cycle time monitoring or no cycle time monitoring is recommended.

Example:

```
VAR
    FtpGetFile              :       FTP_GetFile;
    xFtpGetFil              :       BOOL;
    sSrcFilename            :       STRING := '/GetOut.txt';
    sDstFilename            :       STRING := '\GetIn.txt';
    dwFtpHandle             :       DWORD;
    dwLastError             :       DWORD;
END_VAR

IF xFtpGetFile THEN
    xFtpGetFile := FALSE;
    FtpGetFile  (
                hFtpOpen        := dwFtpHandle,
                psSrcFile       := ADR(sSrcFilename),
                psDstFile       := ADR(sDstFilename),
                xError          => ,
                wErrorId        =>
            );
    IF FtpGetFile.xError THEN
        (* FtpGetFile Error *)
        dwLastError := FTP_GetLastError( TRUE );
    END_IF;
END_IF;
```

2.5.7 FTP_PutFile (FB)

This function block transfers a local file from the FTP client to the FTP server and stores it on the FTP server.



Input parameter	Data type	Description
hFtpOpen	DWORD	Handle of the FTP connection (Output parameter *hFtpOpen* of FB *FTP_OpenConnection*)
psSrcFile	POINTER TO STRING	Memory location (ADR(*xxx*)) of the string with the file name which is local and which can be transferred. e.g. '\Temp\PutOut.txt'
psDstFile	POINTER TO STRING	Memory location (ADR(*xxx*)) of the string with the file name which is transferred to the FTP server. e.g. '/FtpServer/PutIn.txt'

Output parameter	Data type	Description
xError	BOOL	FALSE … No error occured TRUE … Action abort by error
wErrorId	WORD	Additional error code Possible error codes are to be taken from the appendix (➔ Chap. **Fehler! Verweisquelle konnte nicht gefunden werden.**)

➔ This function block is called, processed and finished in a program cycle (similar to a function).

➔ This function block is not real time compatible. The processing of the function block can take some time.

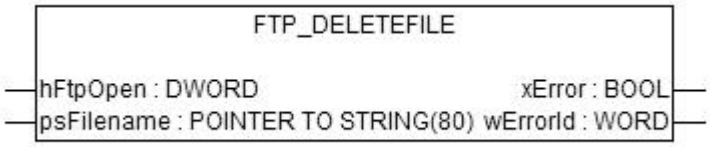
For using the function block a low priority task with a low cycle time monitoring or no cycle time monitoring is recommended.

Example:

```
VAR
    FtpPutFile              :       FTP_PutFile;
    xFtpPutFile             :       BOOL;
    sSrcFilename            :       STRING := '\PutOut.txt';
    sDstFilename            :       STRING := '/PutIn.txt';
    dwFtpHandle             :       DWORD;
    dwLastError             :       DWORD;
END_VAR

IF xFtpPutFile THEN
    xFtpPutFile := FALSE;
    FtpPutFile  (
                hFtpOpen        := dwFtpHandle,
                psSrcFile       := ADR(sSrcFilename),
                psDstFile       := ADR(sDstFilename),
                xError          => ,
                wErrorId        =>
            );
    IF FtpPutFile.xError THEN
        (* FtpPutFile Error *)
        dwLastError := FTP_GetLastError( TRUE );
    END_IF;
END_IF;
```

2.5.8 FTP_DeleteFile (FB)

This function block deletes a file on the FTP server.



Input parameter	Data type	Description
hFtpOpen	DWORD	Handle of the FTP connection (Output parameter *hFtpOpen* of FB *FTP_OpenConnection*)
psFilename	POINTER TO STRING	Memory (ADR(*xxx*)) location of the string with the file name which is on the FTP server and which can be deleted. e.g. '/FtpServer/PutIn.txt'

Output parameter	Data type	Description
xError	BOOL	FALSE … No error occured TRUE … Action abort by error
wErrorId	WORD	Additional error code Possible error codes are to be taken from the appendix (➔ Chap. **Fehler! Verweisquelle konnte nicht gefunden werden.**)

➔ This function block is called, processed and finished in a program cycle (similar to a function).

➔ This function block is not real time compatible. The processing of the function block can take some time.

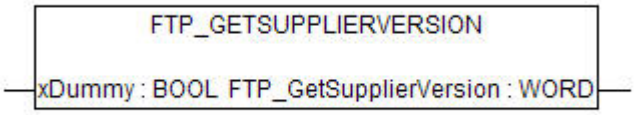
For using the function block a low priority task with a low cycle time monitoring or no cycle time monitoring is recommended.

Example:

```
VAR
    FtpDeleteFile           :       FTP_DeleteFile;
    xFtpDeleteFile          :       BOOL;
    sFilename               :       STRING := '/PutIn.txt';
    dwFtpHandle             :       DWORD;
    dwLastError             :       DWORD;
END_VAR

IF xFtpDeleteFile THEN
    xFtpDeleteFile := FALSE;
    FtpDeleteFile   (
                        hFtpOpen        := dwFtpHandle,
                        psFilename      := ADR(sFilename),
                        xError          => ,
                        wErrorId        =>
                    );
    IF FtpDeleteFile.xError THEN
        (* FtpDeleteFile Error *)
        dwLastError := FTP_GetLastError( TRUE );
    END_IF;
END_IF;
```

2.5.9 FTP_GetSupplierVersion (FUN)

This function returns the current library version.

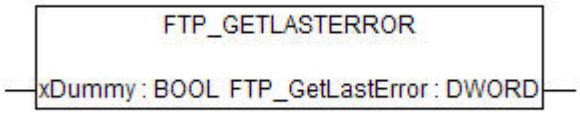


Input parameter	Data type	Description
xDummy	BOOL	Dummy input parameter

Return value	Data type	Description
	WORD	Library version HighByte ... Release version LowByte ... Implemented version

2.5.10 FTP_GetLastError (FUN)

This function returns the system-specific error code of the last incorrect executed action.



Input parameter	Data type	Description
xDummy	BOOL	Dummy input parameter

Return value	Data type	Description
	DWORD	System-specific error code

Possible error codes (function return value):

Value	Designation	Description
1	-	Wrong command execution in the driver
2	-	Temporary files could not be deleted

2.6 Data types

Data type	Description
-	-

2.7 Global variables

Global variable	Data type	Description
FTP_gc_byVersion	BYTE	Constant 16#1 ... Specification version
FTP_gc_Version_1	BYTE	Constant with assigned variable name according to the constant value of the specification version

3 Appendix

Possible error codes (output parameter wErrorId):

Value	Designation	Description
1	FTP_NO_HOST_NAME	No hostname defined
2	FTP_CONNECT_ERROR	Connection to the FTP server could not be opened
3	FTP_WRONG_HANDLE	Wrong handle
4	FTP_NO_FILE_ADDRESS	No file name defined
5	FTP_GET_FILE_ERROR	File could not be read
6	FTP_PUT_FILE_ERROR	File could not be written
7	FTP_CHANGE_DIR_ERROR	Directory could not be changed
8	FTP_FIND_FIRST_FILE	First directory entry could not be read
9	FTP_FIND_NEXT_FILE	Next directory entry could not be read
10	FTP_DELETE_FILE_ERROR	File could not be deleted
11	FTP_TO_MANY_OPEN_CONNECTION	Too many FTP connections opened (max. 100)